

Utility di compressione di immagini tramite DCT

Benchmark e comparatore della compressione DCT2

Nicolas Chines (matr. 899536)
Jacopo Borgato (matr. 866305)

A.A.: 2025/2026

Sommario

Il progetto consiste nella realizzazione di due software che implementano la compressione delle immagini basata su DCT.

Nella prima parte viene implementato l'algoritmo DCT2, confrontando successivamente i tempi di esecuzione con quello implementato dalla libreria numpy.

Per la seconda parte il programma permette all'utente di scegliere da filesystem un'immagine bitmap in toni di grigio, applicare la compressione secondo i parametri specificati e visualizzare affiancate l'immagine originale e compressa.

Contents

1	Introduzione	3
2	Implementazione DCT / DCTN	3
2.1	Implementazione DCT1	3
2.2	Implementazione DCT2	4
3	Benchmark	4
3.1	Implementazione	4
3.2	Analisi dei risultati	5
4	Compressione di Immagini	7
4.1	Interfaccia Grafica	7
4.2	Funzioni GUI	7
4.3	Implementazione	8
4.4	Vari test	10
4.4.1	Fenomeno di Gibbs	10

1 Introduzione

La funzione `dct2` calcola la trasformata discreta del coseno 2D di una matrice 2D, tipicamente un'immagine. Questa trasformata converte i dati spaziali in componenti di frequenza. È una tecnica fondamentale per la compressione di immagini, alla base dello standard JPEG.

L'operazione è separabile: equivale a applicare una DCT 1D prima sulle righe e poi sulle colonne (o viceversa). I coefficienti ottenuti permettono di ricostruire l'immagine originale tramite la trasformata inversa (`idct2`). Durante la compressione, i coefficienti ad alta frequenza, che risultano essere meno percepibili all'occhio umano, possono essere eliminati per ridurre la dimensione del file con una perdita di qualità limitata.

2 Implementazione DCT / DCTN

L'idea che sta alla base del calcolo della DCT è la trasformazione da base canonica in uno spazio vettoriale a quella dei coseni.

2.1 Implementazione DCT1

Come fattore di normalizzazione viene usato $1/\sqrt{N}$ per il primo coefficiente, per i restanti invece $\sqrt{2/N}$.

In fase di inizializzazione viene creato un vettore di output `y`, di lunghezza pari alla dimensione del vettore, pieno di zeri.

```
1 N = len(v)
2 y = np.zeros(N, dtype=float)
```

Successivamente viene calcolato il coefficiente `k` per ogni posizione nel seguente modo:

```
1 if k == 0:
2     Ck = 1.0 / math.sqrt(N)
3 else:
4     Ck = math.sqrt(2.0 / N)
5
6 for i in range(N):
7     ak = math.cos((math.pi * k * (2*i + 1)) / (2 * N)) * v[i]
8     sum_cos += ak
9     y[k] = Ck * sum_cos
```

Dove `Ck` è il fattore di normalizzazione, `ak` è la base della DCT, che rappresenta le diverse frequenze, la quale viene aggiunta alla somma dei coseni.

In questo modo non si ha più informazione locale come nella base canonica ma si ha informazione riguardante tutto il vettore.

Il risultato è quindi il vettore `y` di coefficienti che rappresenta l'input in termini di componenti di frequenza del coseno.

2.2 Implementazione DCT2

Per calcolare la funzione DCT2 é possibile iterare su righe e colonne la DCT1.

```
1 # DCT1 for columns
2 for j in range(N):
3     y_m[:,j] = dct(m_in[:,j])
4
5 # DCT1 for rows
6 for j in range(N):
7     y_m[j,:] = np.transpose(dct(np.transpose(m_in[j,:])))
```

Il primo passaggio è scorrere ogni colonna della matrice originale applicando la funzione `dct1`, salvando il risultato nella corrispondente colonna. Successivamente si applica lo stesso passaggio, alla matrice ottenuta precedentemente, ma sulle righe.

3 Benchmark

3.1 Implementazione

Il file `benchmark.py` implementa un semplice benchmark sulla funzione `dct2` implementata, su matrici di dimensione $N \times N$. con N che ha range da 10 a 200 in step da 10. Confrontandola con quella della libreria Python `numpy`

```
n_values = np.arange(10, 201, 10)
```

Per poi mostrare i risultati a schermo tramite la libreria `matplotlib`.

```
1 for n in n_values:
2     x = np.random.randint(0, 255, (n, n)).astype(float) -
        128
3
4     start_t = time.perf_counter()
5     _ = dctn(x, type=2, norm='ortho')
6     end_t = time.perf_counter()
7     scipy_times.append((end_t - start_t) * 1000) #
        Convertiamo in ms
8
9     start_t = time.perf_counter()
10    _ = utils.dct2(x)
11    end_t = time.perf_counter()
12    custm_times.append((end_t - start_t) * 1000) #
        Convertiamo in ms
13
14    print(f"Test completato per N={n}")
15
16 return np.array(scipy_times), np.array(custm_times)
```

La funzione esegue quindi i seguenti step per ogni valore nel range definito:

1. Genera una matrice quadrata $n \times n$ di valori interi casuali tra 0 e 255 (simulando pixel di un'immagine)
2. Viene applicata la funzione `dctn` della libreria, tenendo traccia del tempo di esecuzione, usando i parametri `norm='ortho'` che applica una normalizzazione ortonormale. e `type=2` che utilizza lo standard per la compressione.
3. Viene misurato il tempo della funzione `dct2` implementata non dalla libreria sulla stessa matrice
4. Infine vengono salvati i tempi di esecuzione per essere successivamente mostrati

3.2 Analisi dei risultati

La figura 1 mostra, su scala logaritmica, i risultati dei vari benchmark su matrici di dimensioni diverse.

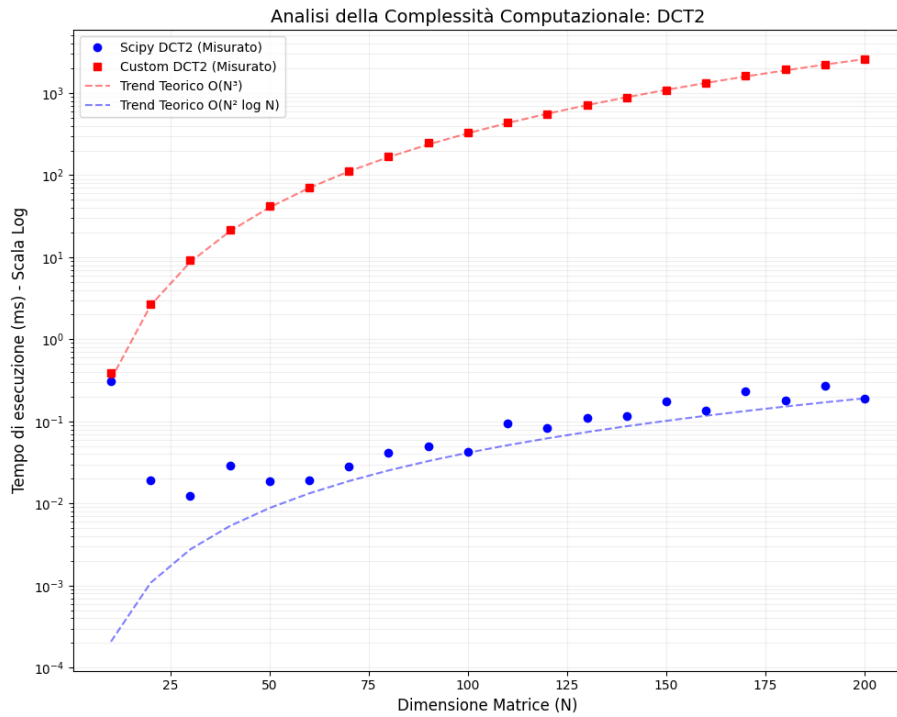


Figure 1: Grafico dei tempi di computazione

Si può notare come l'andamento iniziale sia relativamente irregolare per quanto riguarda l'implementazione della libreria.

Mentre la funzione custom, in mancanza di ottimizzazioni, risulta attenersi di più a $O(N^3)$.

4 Compressione di Immagini

Per la seconda parte del progetto si è implementato un programma dotato di GUI che permetta di mostrare a schermo un'immagine selezionata compressa.

4.1 Interfaccia Grafica

La figura 2 riporta l'interfaccia grafica mostrata quando si esegue il programma senza argomenti, presentando:

- In alto un'area per poter scegliere il file immagine e i rispettivi parametri per la DCT2
- Due aree in basso affiancate per mostrare vicine l'immagine selezionata e la relativa compressa

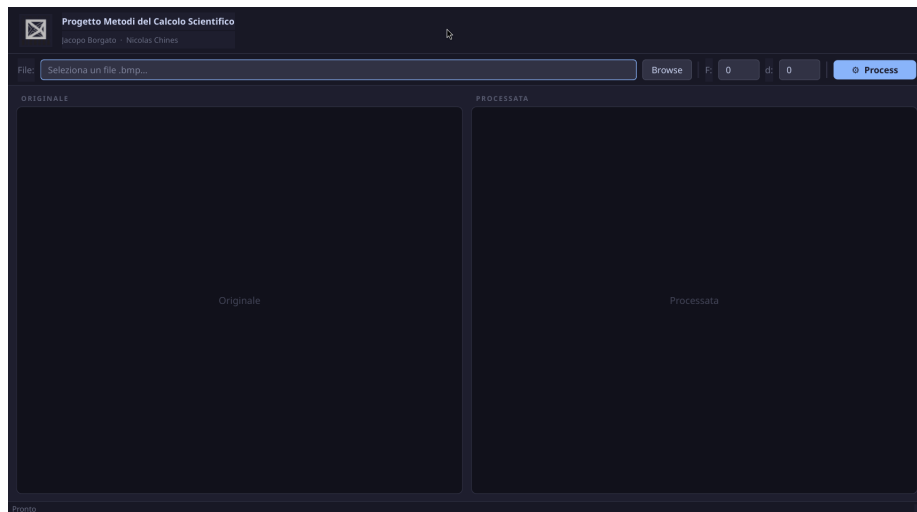


Figure 2: Grafico dei tempi di computazione

L'applicazione è stata scritta utilizzando la libreria Python `PyQT6`, in modo da creare rapidamente un'interfaccia grafica moderna.

Per la gestione delle immagini si è utilizzata la libreria `Pillow`, la quale permette di gestire in modo semplice le immagini.

4.2 Funzioni GUI

Se avviata senza argomenti (2) l'applicazione richiede all'utente di inserire un percorso che punta a un file immagine, in formato `bmp` (Bitmap).

L'immagine deve essere in toni di grigio, se non fosse così viene convertita automaticamente.

```
img = Image.open(path).convert("L")
```

I campi `F` e `d` permettono di impostare relativamente i parametri `F` e `d` relativi alla DCT2.

Dove `F` è la grandezza della matrice $F \times F$ su cui viene applicata la funzione `dctn` dopo aver diviso l'immagine in blocchi.

Mentre `d` rappresenta il valore della soglia delle frequenze da eliminare al fine della compressione.

La figura 3 mostra un esempio di compressione dell'immagine utilizzando una divisione in blocchi da 8×8 mantenendo solamente la prima frequenza (`d=1`).

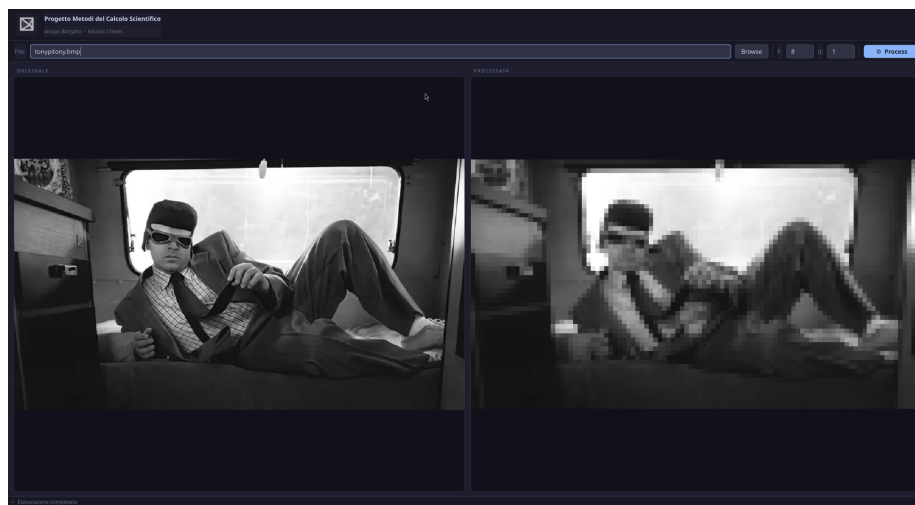


Figure 3: Esempio DC 8x8

4.3 Implementazione

Una volta importata l'immagine Bitmap e convertita in toni di grigio, viene compressa e salvata, rappresentata in bytes, in un buffer, quindi mostrata a fianco dell'originale.

```
1 compressed = jpeg.compress_image(img, F, d)
2 buf = BytesIO()
3 compressed.save(buf, format="BMP")
4 return buf.getvalue()
```

La logica di compressione è implementata nel file `jpeg.py`. In particolare la funzione `compress_image` 1 è quella che implementa la compressione dato un oggetto `Image.Image`.

Listing 1: `compress_image`


```

1 def compress_image(img: Image.Image, F: int, d: int) ->
  Image.Image:
2     """Compress a grayscale PIL image with DCT2 block
      compression using multiple CPU cores."""
3     A = np.array(img.convert("L")) # convert to grayscale
4     n_y, n_x, blocks = block_splitter(A, F)
5     core = os.cpu_count() or 1
6     max_cores = max(1, core - 2) #lasciamo 2 core liberi per
      il sistema e altre operazioni( se meno di cpu.count
      () < 4 core allora ne usa 1)
7     with ProcessPoolExecutor(max_workers=max_cores) as
      executor:
8         compressed_iter = executor.map(
9             compress_block,
10            blocks,
11            [d] * len(blocks)
12        )
13        compressed = list(compressed_iter)
14        rows = [np.hstack(compressed[by * n_x : (by + 1) * n_x])
15                for by in range(n_y)]
16    return Image.fromarray(np.vstack(rows))

```

La funzione inizializza A come array numpy dell'immagine passata (3), divide quindi la matrice (array A) in blocchi da $F \times F$ (4) ed esegue la compressione su ognuno di essi, parallelizzando l'operazione.

Infine ricostruisce l'immagine dai blocchi compressi, prima chiamando la funzione numpy `hstack` (14), che concatena gli array orizzontalmente, e successivamente `hstack` (15), che invece concatena gli array in verticale, ottenendo così l'immagine completa.

La funzione `block_splitter` 2 implementa la logica di suddivisione della matrice rappresentante l'immagine in blocchetti, allineati cominciando dall'alto a sinistra.

Listing 2: `block_splitter`

```

1 def block_splitter(A: np.ndarray, F: int) -> tuple[int, int,
  list[np.ndarray]]:
2     """Split a 2-D array into FxF blocks (top-left aligned,
      remainder discarded).
3     Returns (n_blocks_y, n_blocks_x, list_of_blocks).
4     """
5     h, w = A.shape
6     n_y, n_x = h // F, w // F
7
8     #crop the input array A to ensure its dimensions are
      multiples of F.
9     A = A[: n_y * F, : n_x * F].astype(float)
10
11    #extract the FxF block from the cropped array A for each
      block position (by, bx).

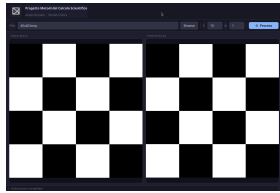
```

```

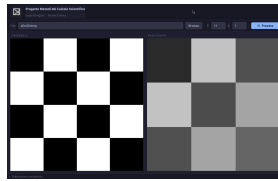
12     return n_y, n_x, [
13         A[by * F : (by + 1) * F, bx * F : (bx + 1) * F]
14         for by in range(n_y)
15         for bx in range(n_x)
16     ]

```

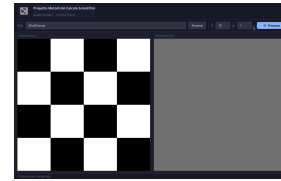
4.4 Vari test



(a) Quadrati da 10



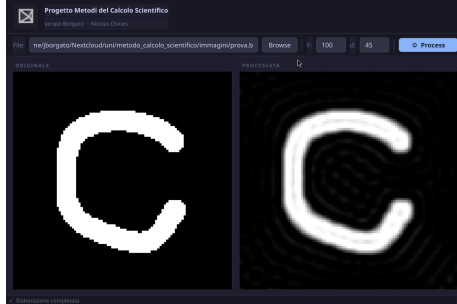
(b) Quadrati da 11



(c) Quadrati da 15

Figure 4: Test su immagine divisa in quadrati

4.4.1 Fenomeno di Gibbs



(a) Gibbs



(b) Dettaglio Gibbs

Figure 5: Fenomeno di Gibbs